Klausur Datenbanksysteme, Lösungen

Aufgaben **(Punkte ohne Gewähr)**

An einer Hochschule werden viele Entscheidungen von Gremien vorbereitet und getroffen, z. B. vom Fachbereichsrat oder vom Prüfungsausschuss. Für die Verwaltung der Gremien und deren Arbeit soll eine Datenbank erstellt werden. Sie sehen auf dem letzten Blatt dazu Abbildung 2: Ausschnitt aus einem Entity-Relationship-Diagramm der Gremien-Datenbank.

Erklärung des Ausschnitts:

Personen sind Studenten, Professoren, Mitarbeiter, Lehrbeauftragte, Industrievertreter usw., die Funktionen an der Fachhochschule wahrnehmen. Neben den angegebenen Attributen *ID*, *Name* und *Adresse* besitzen sie noch weitere Attribute, die in der Datenbank gespeichert werden.

Zentrale Entity sind die **Gremien**. Es gibt *offizielle* Gremien, die eine gesetzliche Grundlage haben, wie z. B. der Hochschulrat, der Senat, der Fachbereichsrat des Fachbereichs Maschinenbau oder der Prüfungsausschuss für Angewandte Informatik. Daneben gibt es auch inoffizielle Gremien, die für besondere Aufgaben eingesetzt werden, z. B. SPO-Kommission für einen Master in Informatik. *Start* gibt an, wann ein Gremium eingesetzt wurde, *Ende* gibt an, wann es seine Arbeit einstellen wird oder eingestellt hat. Bei Gremien, die auf Dauer angelegt sind, bleibt *Ende* leer.

Personen **sind Mitglieder** in Gremien. Dies ist eine mehrdeutige Relation. In dieser Relation wird nur der momentane Zustand gespeichert. Eine historische Entwicklung wer wann Mitglied eines Gremiums war, ist nicht vorgesehen. *Funktion* gibt an, welche Funktion ein Mitglied in einem Gremium ausübt, z. B. Vorsitzende(r).

Die Arbeit der Gremien wird vorwiegend in **Sitzungen** durchgeführt. *Beginn* speichert Datum und Uhrzeit, *Einladung am* gibt das Datum an, an dem die Einladung zur Sitzung verschickt wurde, *öffentlich* ist ein binäres Attribut, das angibt, ob die Sitzung öffentlich ist oder nur für die Teilnehmer zugänglich. Die N:1 Relation Sitzung **des** Gremiums gibt an, zu welchem Gremium eine Sitzung gehört. Sitzungen mehrere Gremien zusammen sind in der Datenbank nicht vorgesehen.

Personen **nehmen** an Sitzungen **teil**. Dies ist eine mehrdeutige Relation. Es können auch Personen an einer Sitzung teilnehmen, die nicht Mitglied des zugehörigen Gremiums sind, andererseits nehmen nicht immer alle Mitglieder eines Gremiums an allen Sitzungen teil.

Bei jeder Sitzung **ist** eine Person **Protokollführer**. Mehrere Protokollführer bei einer Sitzung sind nicht vorgesehen.

Aufgabe 1: Ergänzung des Entity-Relationship-Diagramms **(34)**

Das ER-Diagramm ist noch nicht vollständig. Ergänzen Sie das ER-Diagramm in Abbildung 2 um weitere Entities und Relationen. Achten sie darauf, ...

- dass Sie ein ER-Diagramm erstellen sollen und nicht ein Diagramm einer relationalen Tabellenstruktur, d.h. M:N-Relationen sind noch nicht in Verbindungstabellen umgewandelt, es gibt mehrwertige Attribute, und eine Strukturierung in Unterentities ist möglich.
- dass Sie Anomalien und Redundanzen vermeiden.
- dass Sie die Art der Relationen in der Ausführlichen (min, max)-Schreibweise kennzeichnen.

Folgende Informationen sollen aus der Datenbank effizient abgefragt werden können (Informationen, die hier nicht genannt sind, brauchen auch nicht ins ER-Diagramm, selbst wenn Sie sie für eine Datenbank über Gremien für sinnvoll erachten.):

- Suche nach allen **Tagesordnungspunkten** einer Sitzung. Jeder Tagesordnungspunkt hat einen Titel, eine Kurzbeschreibung und (nach der Sitzung) einen Protokolltext. Eine Sitzung hat normalerweise mehrere Tagesordnungspunkte.

- In den Sitzungen können **Anträge** gestellt werden und darüber abgestimmt werden. Jeder Antrag hat einen Titel, einen Antragstext, das Ergebnis der Abstimmung in Form von Ja-Stimmen, Nein-Stimmen und Enthaltungen sowie einem Attribut *angenommen*. Jeder Antrag muss einem Tagesordnungspunkt zugeordnet sein. Folgende Abfragen sollen möglich sein:
  - Suche nach allen Anträgen zu einem Tagesordnungspunkt.
  - Suche nach allen Anträgen, die eine bestimmte Person gestellt hat.
  - Suche nach allen Personen, die einen bestimmten Antrag gestellt haben.
- Bei den Sitzungen werden **Dokumente** verteilt, z. B. eine Excel-Tabelle über Deputatsstunden der Professoren oder eine Grafik mit verschiedenen Vorschlägen für das FH-Logo. Neben dem eigentlichen Inhalt hat ein Dokument noch einen Mime-Typ (Art des Dokuments, z. B. Pixel-Bild, pdf-Dokument oder Excel-Tabelle) und ein Erstellungsdatum. Folgende Abfragen sollen möglich sein:
  - Suche nach allen Dokumenten zu einem Tagesordnungspunkt.
  - Suche nach allen Tagesordnungspunkten, zu denen ein Dokument gehört.
  - Suche nach allen Autoren eines bestimmten Dokuments.
  - Suche nach allen Dokumenten, die eine bestimmte Person erstellt hat.
- Gremien können Untergremien haben, z. B. ist der Fachbereichsrat ein Untergremium des erweiterten Fachbereichsrats. Tragen Sie die Relation **ist Untergremium von** in das ER-Diagramm ein.
- Jedes Gremium hat ein oder mehrere **Aufgabengebiete**, z. B. hat das Gremium "Öffentlichkeitsarbeit im Fachbereich Elektrotechnik und Informatik" die Aufgabengebiete: "Kontakte zu Schulen pflegen", "Werbebroschüren und Plakate gestalten", "Informationsveranstaltungen planen", "Internetseiten gestalten und pflegen", "Presseartikel verfassen".

Aufgabe 2: Ergänzung der Tabellenstruktur der Gremien-Datenbank (11)

In Aufgabe 1: haben Sie unter anderem Zusatzinformation bezüglich **Tagesordnungspunkten** und **Anträgen** in das Entity-Relationship-Diagramm eingefügt. Wandeln Sie nur die zu **Tagesordnungspunkten** und **Anträgen** eingetragenen Entities und Relationen in Tabellen eines relationalen Datenbanksystems um und tragen diese Tabellen in **Abbildung 1: Tabellenstruktur der Datenbank für Gremien** ein. Kennzeichnen Sie Primär- und Fremdschlüssel analog zu den bereits eingetragenen Tabellen.

Aufgabe 3: SQL-Abfrage (4)

Geben Sie eine SQL-Abfrage an, die die Bezeichnungen aller offizieller Gremien ermittelt, die keinen Endtermin eingetragen haben.

```
SELECT Bezeichnung •
FROM Gremium •
WHERE offiziell = 'Ja' • AND Ende IS NULL •;
```

Aufgabe 4: SQL-Abfrage (9)

Geben Sie dann eine SQL-Abfrage an, die die Namen aller Teilnehmer der Sitzung im Raum H039 mit Beginn = '20.01.2003, 10:00' ermittelt.

```
SELECT Person.Name •
FROM Person •, Teilnahme •, Sitzung •
WHERE Person.Person_Nr = Teilnahme.Person_Nr •• AND
 Teilnahme.Sitzung_ID = Sitzung.ID •• AND
 Sitzung.Beginn = '20.01.2003, 10:00' AND
 Raum = 'H039' •;
```

Aufgabe 5: SQL-Abfrage **(10)**

Geben Sie eine SQL-Abfrage an, mit der alle Personen mit Namen zusammen mit der Bezeichnung des Gremiums ausgegeben werden, die zu dem Gremium gehören, das im Raum H039 eine Sitzung hatte mit Beginn = '20.01.2003, 10:00'.

```
SELECT Person.Name •, Gremium.Bezeichnung •
FROM Person •, Gremium •, Mitglied •, Sitzung •
WHERE Person.Person_Nr = Mitglied.Person_Nr • AND
 Mitglied.Gremium_ID = Gremium.ID • AND
 Sitzung.Gremium_ID = Gremium.ID • AND
 Sitzung.Beginn = '20.01.2003, 10:00' AND
 Raum = 'H039' •;
```

Aufgabe 6: SQL-Abfrage **(8)**

Geben Sie eine SQL-Abfrage an, die für jedes Gremium (gekennzeichnet durch seine ID) die durchschnittliche Anzahl der Teilnehmer bei den Sitzungen dieses Gremiums ermittelt.

```
SELECT Sitzung.Gremium_ID •,
 COUNT(Teilnahme.Person_NR)/COUNT(DISTINCT Teilnahme.Sitzung_ID) ••
FROM Teilnahme •, Sitzung •
WHERE Teilnahme.Sitzung_ID = Sitzung.ID •
GROUP BY Sitzung.Gremium_ID ••;
```

Alternative Lösung

```
SELECT Mitglied.Gremium_ID •,
 COUNT(Teilnahme.Person_NR)/COUNT(DISTINCT Teilnahme.Sitzung_ID) ••
FROM Teilnahme •, Mitglied •
WHERE Teilnahme.Person_NR = Mitglied.Person_Nr •
GROUP BY Mitglied.Gremium_ID ••;
```

Alternative Lösung

```
CREATE VIEW Sitzungsgroesse AS
SELECT COUNT(Person_Nr) Teilnehmerzahl •, Sitzung_ID
FROM Teilnahme GROUP BY Sitzung_ID •;

SELECT Sitzung.Gremium_ID •, AVG(Teilnehmerzahl) •
FROM Sitzungsgroesse •, Sitzung •
WHERE Sitzungsgroesse.Sitzung_ID = Sitzung.Sitzung_ID •
GROUP BY Sitzung.Gremium_ID •;
```

Da diese Aufgabe besonders schwierig war, werden die erreichbaren 8 Punkte nicht bei der Maximalpunktzahl berücksichtigt, d. h. die Maximalpunktzahl sinkt von 120 auf 112. Die persönlich erzielten Punkte in dieser Aufgabe werden aber bei jedem berücksichtigt.

Aufgabe 7: SQL-Anweisungen **(14)**

Es wird ein neues inoffizielles Gremium eingerichtet: 'Kommission zur Planung der SPO für einen Master in Informatik', mit der ID 42, dem Start am 27.01.2003, Ende am 31.08.2005. Mitglieder des Gremiums sind die Personen mit der Person_Nr 10, 25, und 14531, wobei die Person mit der Nr. 25 den Vorsitz hat. Geben Sie SQL-Anweisungen an, mit denen diese Informationen (nur die in dieser Aufgabe genannten Informationen) in die Datenbank eingetragen werden.

```
INSERT • INTO Gremium • VALUES • (42 •, 'Kommission zur Planung der
SPO für einen Master in Informatik' •, 'nein' •, '27.01.2003',
'31.08.2005' •);

INSERT INTO Mitglied • VALUES (10 •, 42 •, NULL •);

INSERT INTO Mitglied VALUES (25 •, 42, 'Vorsitz' •);

INSERT INTO Mitglied VALUES (14531 •, 42, NULL);
```

Aufgabe 8: Boyce-Codd Normalform **(10)**

Ein Student erstellt für die Gremien-Datenbank eine Tabelle **Anträge**. Unten sehen Sie einen Ausschnitt dieser Tabelle.

A_Nr	Antrag_Text	Ja	Nein	Enth	ange-nommen	Person_Nr	Antragstel-ler_Name
…							
11	Prüfungen abschaffen	2	15	1	Nein	99	Peter Angst
11	Prüfungen abschaffen	2	15	1	Nein	100	Sonja Zweifel
12	Vorlesungsbeginn auf 7:00 Uhr vorverlegen	10	3	5	Ja	100	Sonja Zweifel
…							

Ergänzen Sie folgende funktionelle Abhängigkeiten:

A_Nr → {Antrag_Text, Ja, Nein, Enth, angenommen} ••

Person_Nr → Antragsteller_Name ••

Ist die Tabelle in Boyce-Codd-Normalform?

☐ Ja ☑ Nein •

Begründung:

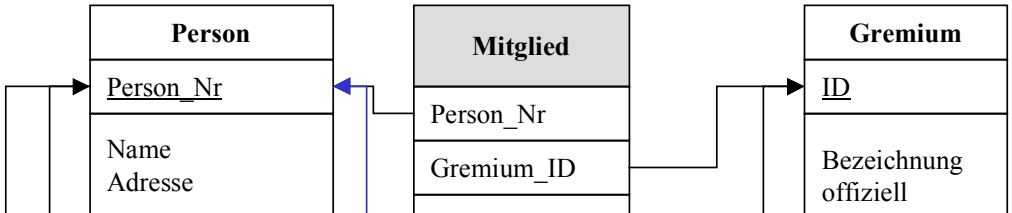
Schlüssel • der obigen Tabelle ist {A_Nr, Person_Nr} •. Auf der linken Seite der beiden funktionellen Abhängigkeiten steht aber nur A_Nr oder Person_Nr, d. h. kein Superschlüssel ••. Dies wäre aber notwendig für die Boyce-Codd-Normalform. •

Aufgabe 9: Trigger **(20)**

Erstellen Sie einen Datenbank-Trigger, der beim Eintrag einer neuen Sitzung oder beim Ändern der Protokollführer-ID automatisch den Protokollführer auch als Teilnehmer der Sitzung einträgt, falls er in der Tabelle Teilnahme noch nicht eingetragen ist.

Hinweis:
Falls Sie die genaue PL/SQL-Syntax einer Funktion oder eines Konstrukts nicht kennen, verwenden Sie C-Syntax oder beschreiben die Operation, die Sie durchführen wollen, in einem Kommentar.

```
CREATE OR REPLACE TRIGGER Protokollführer_als_Teilnehmer_eintragen
BEFORE •    AFTER //nichtzutreffendes streichen
INSERT OR DELETE •      //nichtzutreffendes streichen
OR UPDATE OF Protokollführer •
ON Sitzung • //Tabellenname ergänzen
FOR EACH ROW      FOR EACH STATEMENT •    //nichtzutreffendes streichen

DECLARE
 i INTEGER; •

BEGIN
IF :new.Protokollführer IS NOT NULL THEN •
//tu nur dann etwas, wenn ein Protokollführer eingetragen wird

      SELECT COUNT(Person_Nr) • INTO i •
      FROM Teilnahme •
      WHERE Sitzung_ID = :new.ID • AND Person_Nr = :new.Protokollführer; •
      // i = 0: Protokollführer ist noch nicht als Teilnehmer eingetragen
      // i = 1: Protokollführer ist bereits als Teilnehmer eingetragen

      IF i = 0 THEN •
            INSERT • INTO Teilnahme •• VALUES
                  (:new.Protokollführer ••, :new.ID ••);
            // trage Protokollführer als Teilnehmer ein
      END IF;

END IF;

END;
/
```


Person
Person_Nr
Name
Adresse
Mitglied
Person_Nr
Gremium_ID
Gremium
ID
Bezeichnung
offiziell

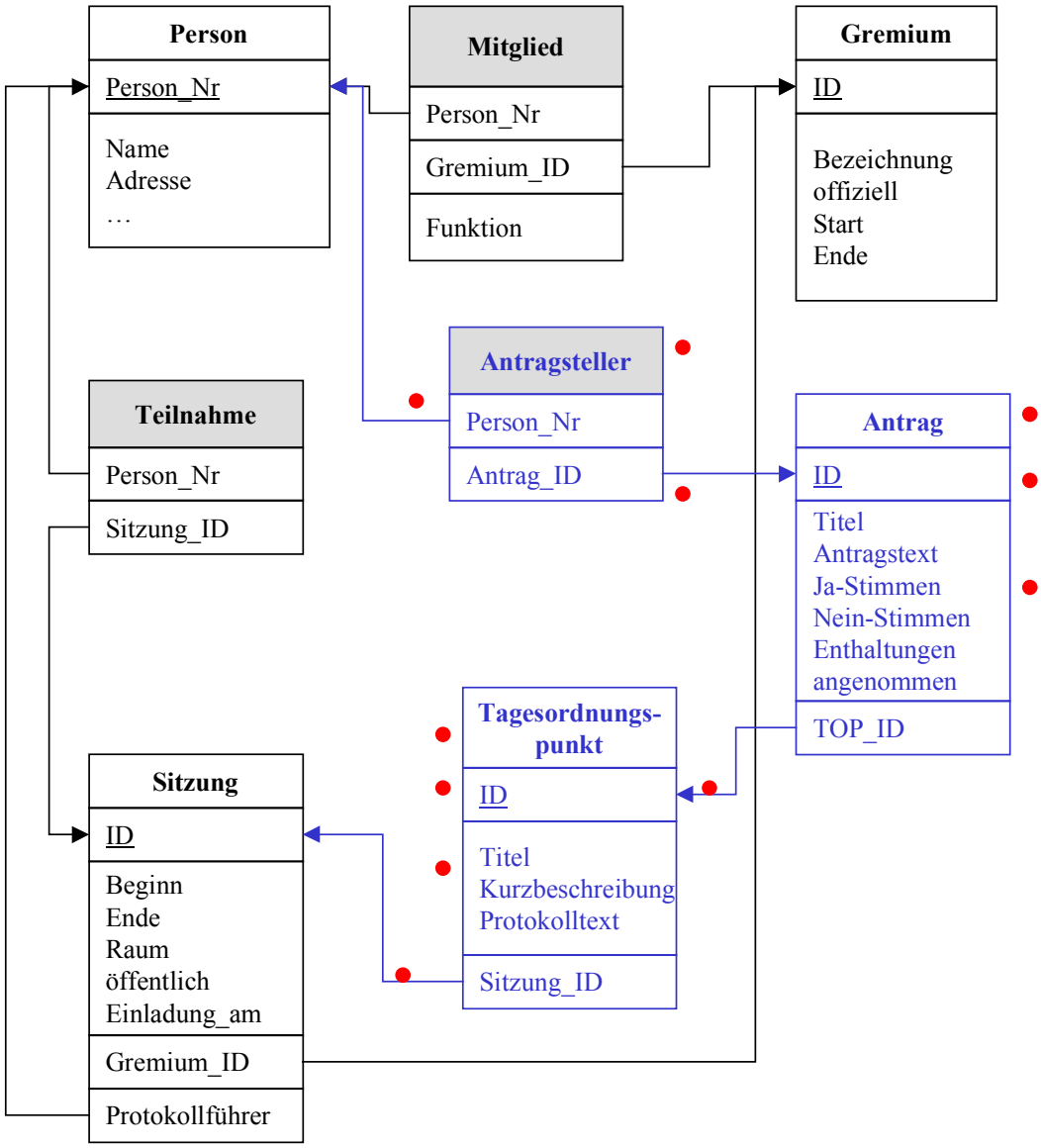


Abbildung 1: Tabellenstruktur der Datenbank für Gremien

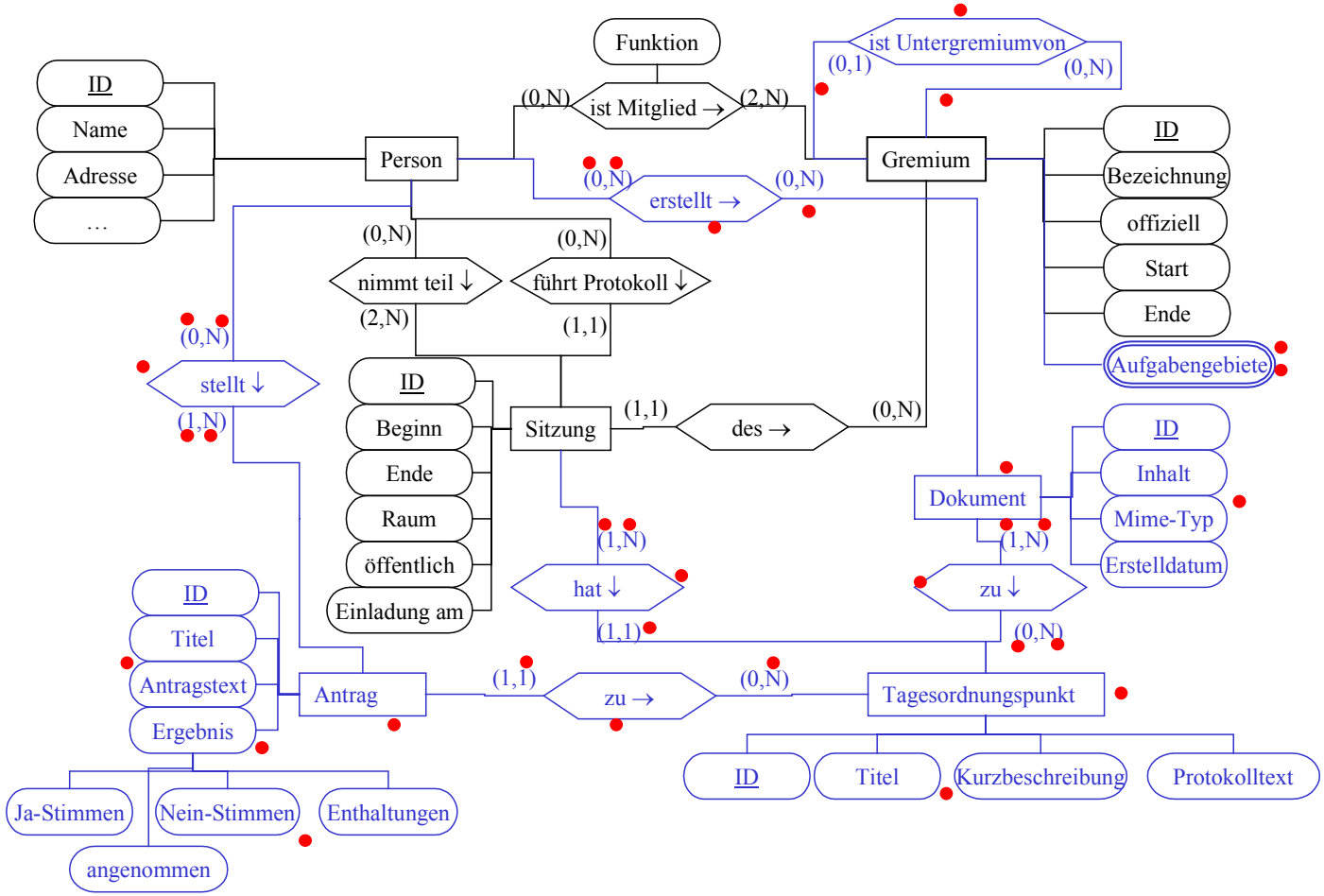


Abbildung 2: Ausschnitt aus einem Entity-Relationship-Diagramm der Gremien-Datenbank